# mc6801-tools User's Manual

Assembler, linker, archiver and utilities for the Motorola MC6801

October 2026

# Contents


**1 Introduction** **4**

1.1 Why a linker? . . . 4

1.2 Typical workflow . . . 4

1.3 Heritage . . . 5

**2 Building and Installing** **6**

2.1 Requirements . . . 6

2.2 Building . . . 6

2.3 Installing . . . 6

2.4 Manual pages . . . 7

2.5 Building the manual . . . 7

**3 Getting Started** **8**

**4 The Assembler** **10**

4.1 Invoking as01 . . . 10

4.2 Source lines . . . 10

4.3 Symbols . . . 11

4.4 Constants . . . 11

4.5 Expressions . . . 11

4.6 Addressing modes . . . 12

4.7 Instruction set . . . 13

4.8 Directives . . . 13

4.8.1 Defining data . . . 13

4.8.2 Defining symbols . . . 13

4.8.3 Location and sections . . . 14

4.8.4 Conditional assembly . . . 14

4.8.5 Macros . . . 15
4.8.6 Other directives . . . 15
4.9 Listings and symbol files . . . 15
4.10 Absolute programs . . . 16
4.11 Error messages . . . 16

**5 Modules and Linking** **18**
5.1 Exporting and importing symbols . . . 18
5.2 Location counters and segments . . . 18
5.3 Invoking ld01 . . . 19
5.4 Output . . . 20
5.5 Start-up modules . . . 20
5.6 Libraries . . . 20
5.7 Linker symbols . . . 20
5.8 Link maps . . . 21

**6 Libraries** **22**

**7 Inspecting Object Files and Executables** **23**

**8 Converting Executables** **25**

**9 A Worked Example** **26**
9.1 Modules . . . 26
9.2 A library . . . 27
9.3 The start-up module and main program . . . 27
9.4 Linking and converting . . . 28

**10 Known Limitations** **29**

**A Instruction Set** **30**

**B File Formats** **33**
B.1 Object files . . . 33
B.2 Executables . . . 33

**C Command Reference** **34**

C.1 as01(1) . . . . . 34

C.2 ld01(1) . . . . . 39

C.3 ar01(1) . . . . . 44

C.4 objdump01(1) . . . . . 46

C.5 objcopy01(1) . . . . . 48

# Chapter 1

# Introduction

mc6801-tools is a set of Unix-style development tools for the Motorola MC6801 and MC6803 microprocessors, and for machines built around them, such as the Tandy MC-10. They run on Linux and other Unix-like systems.

| Tool | Purpose |
|---|---|
| `as01` | Assembler: turns MC6801 assembly source into relocatable object modules or absolute binaries |
| `ld01` | Linker: combines object modules and libraries into an executable |
| `ar01` | Archiver: collects object modules into libraries |
| `objdump01` | Displays the contents of executables; also installed as `nm01` and `size01` |
| `objcopy01` | Converts executables into S-records, Intel hex, raw binaries and cassette images |

## 1.1 Why a linker?

Many 6800-family assemblers are *absolute* assemblers: a program is one source file (perhaps with include files), assembled at a fixed address straight into a loadable image. `as01` can work that way too.

Its main strength, though, is *separate assembly*. You can split a program into modules, each assembled on its own into an object file. Modules export the symbols they define and import the ones they use. The linker then joins them, resolves the references between them and places the whole program at its final address. Commonly used modules can be kept in libraries, and the linker only includes the ones that a program actually needs.

## 1.2 Typical workflow

```
prog.s --as01--> prog.o --+
util.s --as01--> util.o --+--ld01--> prog.aout --objcopy01--> .wav .s19 .hex .bin
```

```
a.o b.o --ar01--> libx.a -+              |
                          +--objdump01--> symbols, sizes, dump
```

## 1.3 Heritage

The tools come from the MC6809 configuration of the assembler, linker and archiver in Bruce Evans' C compiler (`bcc`), as maintained in the *dev86* package for Linux (https://github.com/lkundrak/dev86). They have been adapted for the MC6801 instruction set.

The assembler syntax follows the one described in section C.5 of the *Introl-C Compiler Reference Manual*. The object file format is the *Introl Linkable Binary File Format*, described in the *Introl Linker and Loader Reference Manual*. Both are available at http://www.swtpc.com/mholley/Introl/.

# Chapter 2

# Building and Installing

## 2.1 Requirements

You need a C compiler and `make`. Building this manual also needs `pandoc` and a LaTeX installation (see Section 2.5).

## 2.2 Building

From the top of the source tree:

```
$ make
```

This builds `ar/ar01`, `as/as01`, `ld/ld01`, `ld/objdump01` and `ld/objcopy01`. The build stops at the first error.

The version number, which `as01 -v` and `ld01 -v` print, is taken from the `VERSION` file at the top of the source tree.

## 2.3 Installing

```
$ sudo make install
```

All the programs are installed into a single directory, by default `/usr/local/m6801-tools`. `nm01` and `size01` are created there as links to `objdump01`. The manual pages are installed into `/usr/local/share/man/man1`. To choose other directories, set `PREFIX` for the programs and `MANDIR` for the manual pages; to stage the files for packaging, set `DESTDIR` as well:

```
$ make install PREFIX=/opt/m6801
$ make install DESTDIR=/tmp/pkgroot PREFIX=/usr/bin MANDIR=/usr/share/man
```

Add the installation directory to your `PATH`:

```
$ export PATH=$PATH:/usr/local/m6801-tools
```

## 2.4 Manual pages

Each tool has a manual page in `doc/man`. `make install` installs them (see above), with `nm01` and `size01` as links to the `objdump01` page. To install only the manual pages, use:

```
$ sudo make install-man
$ man as01
```

You can also read one without installing it, using `man -l doc/man/as01.1`.

## 2.5 Building the manual

This manual is built from `doc/manual.md`, with the command reference generated from the manual pages and the instruction table generated by running the assembler:

```
$ make            # build the tools first
$ cd doc
$ make
```

# Chapter 3

# Getting Started

This chapter builds a small program for the Tandy MC-10, which has an MC6803 processor. The program prints a message on the screen using the character output routine in the MC-10 ROM, whose address is stored at `$FFDE`.

Save this as `hello.s`:

```
; hello.s - print a message on the MC-10 screen
        LOC     0
        ENTRY   start

start   ldx     #msg
loop    ldaa    ,x              ; next character
        beq     done            ; zero byte ends the string
        inx
        pshx
        ldx     $ffde           ; ROM character output routine
        jsr     ,x
        pulx
        bra     loop
done    rts

msg     fcc     "HELLO, WORLD!"
        fcb     $0d,0
```

Labels start in column 1; instructions are indented. `ENTRY start` marks `start` as the place where the program begins.

**Assemble** the source into an object module:

```
$ as01 -o hello.o hello.s
```

**Link** it. MC-10 machine language programs are commonly loaded at `$434C`, just above the BASIC program area. The linker's `-T` option sets the start address. Like all addresses given to the tools, it is read like a C number, so write hexadecimal with `0x`:

```
$ ld01 -T0x434c -o hello.aout hello.o
```

**Convert** the result into a cassette audio file. `objcopy01` finds the load address in the executable, and `-n` sets the name that the MC-10 shows when it loads the file:

```
$ objcopy01 -O wav -n HELLO hello.aout hello.wav
```

Play `hello.wav` into the MC-10's cassette input (or load it into an emulator), type `CLOADM`, and then `EXEC` to run the program.

To see what the assembler produced, ask for a listing with `-l`:

```
$ as01 -o hello.o -l hello.lst hello.s
```

```
00005 0000   CE   0012> start   ldx #msg
00006 0003   A6     00  loop    ldaa    ,x      ; next character
00007 0005   27     0A      beq done        ; zero byte ends the string
...
00016 0012           48  msg fcc "HELLO, WORLD!"
```

The columns are the line number, the address, the opcode and the operand bytes. A `>` after the operand means the value is relocatable and will be adjusted by the linker. Here `msg` is at offset `$0012` from the start of the module, and becomes `$435E` once linked at `$434C`. `fcc` and `fcb` lines show only the first byte.

# Chapter 4

# The Assembler

## 4.1 Invoking as01

```
as01 [-gkuvw] [-b binfile] [-l [listfile]] [-m [listfile]]
     [-n name] [-o objfile] [-s symfile] [-t n] [srcfile]
```

| Option | Meaning |
|---|---|
| `-o objfile` | Write a relocatable object file |
| `-b binfile` | Write an absolute binary image (see Section 4.10) |
| `-l [listfile]` | Write a listing to *listfile*, or to standard output |
| `-m [listfile]` | List macro expansions in full (see Section 4.9) |
| `-s symfile` | Write a symbol table file |
| `-n name` | Module name to store in the object file (default: from the source file name) |
| `-g` | Put only global symbols in the object file |
| `-u` | Treat undefined symbols as imported |
| `-w-` | Enable warnings (they are off by default) |
| `-k` | Keep output files even if there are errors |
| `-t n` | Put `.TEXT` sections in location counter $n$+3 |
| `-v` | Print the version and exit |

Each option must be a separate argument (write `-g -u`, not `-gu`). Only one source file may be given; if there is none, the source is read from standard input. If there are errors, the object and binary files are deleted (unless `-k` is given) and `as01` exits with status 1.

## 4.2 Source lines

Each line has up to four fields:

```
[label[:]]   [operation   [operands]]   [; comment]
```

- A **label** without a colon must start in column 1. A label followed by a colon may be indented.
- The **operation** is an instruction mnemonic, a directive or a macro name. It must be preceded by white space unless it follows a label.
- **Operands** are separated by commas.
- A **comment** starts with `;` or `!`. A line whose first character is `*`, `;` or `!` is a comment line. Blank lines are allowed.

A line consisting only of a label defines the label at the current address. If the assembler finds a word it doesn't know where an operation should be, it reports `opcode expected`.

```
* A comment line
table   fcb     1,2,3           ; label in column 1
        ldaa    table           ; instruction, indented
next:   inca                    ; label with colon
here                            ; label alone
```

## 4.3 Symbols

Symbols are made of letters, digits, `_` and `.`, and must not start with a digit. They are case sensitive: `Loop` and `loop` are different.

Mnemonics, directives and register names are recognized in all upper case or all lower case (`LDAA` or `ldaa`, but not `LdAa`). They cannot be used as labels. The register names are `A`, `B`, `D` and `X`.

## 4.4 Constants

| Form | Meaning |
|---|---|
| `123` | Decimal |
| `$7F` or `0x7F` | Hexadecimal |
| `0177` | Octal (any number with a leading zero) |
| `%1010` | Binary |
| `'A` or `'A'` | Character code |

## 4.5 Expressions

From lowest to highest precedence, the expression operators are:

| Operators | Meaning |
|---|---|
| `= < >` | Comparison: true is -1 (`$FFFF`), false is 0 |
| `+ - \|` | Addition, subtraction, bitwise OR |
| `* / & << >>` | Multiplication, division, bitwise AND, shifts |
| `- + ~` | Unary minus, unary plus, bitwise complement |

Use parentheses to group subexpressions. A `*` where an operand is expected stands for the current location counter, so `bra *` is an infinite loop.

A symbol is *absolute* if it has a fixed value (a constant, or a label after `ORG` in a non-relocatable program) or *relocatable* if its value depends on where the linker puts the module. You can add a constant to a relocatable value or subtract two relocatable values from the same section, but other combinations, such as adding two relocatable values, are errors.

## 4.6 Addressing modes

| Mode | Example | Notes |
|---|---|---|
| Inherent | `inx` | No operand |
| Immediate | `ldaa #$10` | `#` prefix; 16-bit for `LDD`, `LDX`, `LDS`, `CPX`, `ADDD`, `SUBD` |
| Direct | `ldaa $80` | Address in `$00`–`$FF`, one byte |
| Extended | `ldaa $1234` | Any 16-bit address |
| Indexed | `ldaa 5,x` | Unsigned 8-bit offset (0–255) from X; `,x` means `0,x` |
| Relative | `bne loop` | Branches; target within -128 to +127 bytes |

The assembler chooses between direct and extended addressing on its own. It uses direct addressing when the address is known in the first pass and is in `$00`–`$FF`. Forward references and relocatable addresses always use extended addressing. You can force the choice by writing `<` (direct) or `>` (extended) before the operand:

```
        ldaa    <port           ; direct, even if port is defined later
        ldaa    >$0080          ; extended, even though $80 fits in a byte
```

The MC6801 has no direct mode for the read-modify-write instructions (`NEG`, `COM`, `LSR`, `ROR`, `ASR`, `ASL`/`LSL`, `ROL`, `DEC`, `INC`, `TST`, `CLR`) or for `JMP`. These always use extended addressing for non-indexed operands, and `<` is an error with them.

The MC6801 has 128 bytes of internal RAM at `$0080`–`$00FF` and its I/O registers at `$0000`–`$001F`. Define symbols for them *before* using them, so that the assembler can use direct addressing:

```
PORT1   equ     $02
RAM     equ     $80

        ldaa    PORT1           ; direct: 96 02
        clr     RAM             ; extended: 7F 0080 (no direct CLR)
```

## 4.7 Instruction set

`as01` accepts the full MC6801/MC6803 instruction set, using the mnemonics from the Motorola data sheet. Appendix A lists every instruction with its opcodes. Some instructions have two names:

| Name | Same as |
|---|---|
| `CMPX` | `CPX` |
| `LSL`, `LSLA`, `LSLB`, `LSLD` | `ASL`, `ASLA`, `ASLB`, `ASLD` |
| `BHS` | `BCC` |
| `BLO` | `BCS` |

## 4.8 Directives

Directives (also called pseudo-ops) control the assembler. Many have several names, for compatibility with other assemblers; they are separated by `/` below.

### 4.8.1 Defining data

| Directive | Meaning |
|---|---|
| `FCB`/`.BYTE`/`DB`/`.DATA1` *expr*,... | Bytes |
| `FDB`/`.WORD`/`DW`/`.SHORT`/`.DATA2` *expr*,... | 16-bit words, high byte first |
| `FCC`/`.ASCII` *string* | Characters of a string |
| `.ASCIZ` *string* | Characters of a string, then a zero byte |
| `RMB`/`.BLKB`/`.SPACE` *count*[,*fill*] | Reserve *count* bytes; filled with *fill* if given |
| `.BLKW`/`.ZEROW` *count* | Reserve *count* words |

A string starts with a delimiter character and ends at the next occurrence of it, so `"text"`, `/text/` and `'text'` are all valid. Two delimiters in a row stand for one delimiter character. These C escape sequences are recognized: `\n`, `\r`, `\t`, `\a`, `\b`, `\f`, `\v`, `\\`, `\'`, `\"`, `\?` and octal `\0` to `\377`.

```
msg     fcc     "Hello"
        fcb     $0d,0
name    .asciz  /MC-10/
table   fdb     start,stop,$1000
buffer  rmb     32              ; 32 bytes, not initialized
spaces  rmb     8,$20           ; 8 bytes containing $20
```

### 4.8.2 Defining symbols

| Directive | Meaning |
|---|---|
| *label* `EQU` *expr* | Define a constant |

| Directive | Meaning |
|---|---|
| *label* `=` *expr* | Same as `EQU` |
| *label* `SET` *expr* | Define a symbol that can be redefined later |
| `EXPORT`/`PUBLIC`/`.DEFINE` *sym*,... | Make symbols visible to other modules |
| `ENTRY` *sym* | Export *sym* and mark it as the program entry point |
| `IMPORT`/`EXTERN`/`.EXTERN`/`EXTRN` *sym*,... | Use symbols from other modules |
| `.GLOBL`/`.GLOBAL` *sym*,... | Export if defined here, otherwise import |
| *label* `COMM` *size* | Common block of *size* bytes, shared between modules |
| *label* `LCOMM` *size* | Common block local to this module |
| `.COMM`/`.LCOMM` *name*,*size* | Unix-style forms of the above |

### 4.8.3 Location and sections

| Directive | Meaning |
|---|---|
| `LOC` *n* | Switch to location counter *n* (0–15) |
| `.TEXT` | Switch to location counter 0 |
| `.DATA`, `.ROM`, `.BSS` | Switch to location counter 3 |
| `.SECT` *name* | Switch to section `.text`, `.data` or `.bss` |
| `ORG` *expr* | Set the location counter to *expr* |
| `BLOCK` *expr* ... `ENDB` | Assemble the enclosed lines at *expr*, then return to the previous location |
| `.ALIGN` *n* | Advance to a multiple of *n* |
| `.EVEN` | Advance to an even address |

Section 5.2 explains how the location counters are mapped into memory.

### 4.8.4 Conditional assembly

| Directive | Meaning |
|---|---|
| `IF` *expr* | Assemble the following lines if *expr* is non-zero |
| `IFC` (*str1*,*str2*) | Assemble the following lines if the strings are identical |
| `ELSEIF` *expr*, `ELSEIFC` (*str1*,*str2*) | Alternative conditions |
| `ELSE` | Assemble if no earlier condition was true |
| `ENDIF` | End of the conditional block |

Conditionals may be nested. Conditions should only use symbols defined earlier in the source.

```
DEBUG   equ     1
```

```
IF      DEBUG
jsr     dump_registers
ELSE
nop
ENDIF
```

### 4.8.5 Macros

A macro is defined with `MACRO` *name* and ends with `MEND`. Inside the body, `?1`, `?2`, ... are replaced by the arguments of the call. The arguments are separated by commas.

```
MACRO   store2          ; store A and B at two addresses
staa    ?1
stab    ?2
MEND

store2  $1000,$1002
```

A macro must be defined before it is used. Within a macro, `IFC` can test the arguments, for example `IFC (?1,)` to check whether the first argument is empty.

### 4.8.6 Other directives

| Directive | Meaning |
|---|---|
| `INCLUDE`/`GET` *file* | Assemble the contents of *file* at this point |
| `END` | End of the source; anything after it is ignored |
| `FAIL` | Report an error (useful in conditionals) |
| `.LIST`, `.MACLIST` [*n*] | Turn listing or macro listing on and off (only when `-l` or `-m` is used) |
| `.WARN` [*n*] | Turn warnings on (`.WARN 1`) and off again (`.WARN`) |

## 4.9 Listings and symbol files

`-l` writes a listing with the source, the address and the generated bytes, followed by a symbol table and the number of errors and warnings (see the example in Chapter 3). The instructions generated by macro calls are listed after the call, marked with `+a`.

`-m` lists every line of each macro expansion, including directives and lines skipped by conditionals. On its own it lists only the macro expansions; use `-l -m listfile` to get the full listing as well.

In the listing's symbol table, each symbol is followed by its location counter number, its value, `A` (absolute) or `R` (relocatable), and `E` (exported), `I` (imported), `N` (entry point) or `C` (common).

`-s` writes a symbol file with one line per symbol: the counter, the value, five flag columns (`E` exported, `N` entry, `I` imported, `R` or `A`, `C` common) and the name:

```
0 0000 --IR- ext
0 000C EN-R- here
```

## 4.10 Absolute programs

For a simple program you can skip the linker entirely. Use `ORG` to set the address and `-b` to write the binary image:

```
        org     $e000
reset   lds     #$00ff
        ...
        org     $fffe
        fdb     reset           ; reset vector
```

```
$ as01 -b rom.bin rom.s
```

The image starts at the lowest address assembled and ends at the highest, so this example produces an 8 KB file covering `$E000` to `$FFFF`. Gaps between `ORG` areas are filled with zeros. `IMPORT` and other external references are not allowed with `-b`.

## 4.11 Error messages

Errors are shown in the listing under the line that caused them, with a `^` pointing at the position. If no listing file was requested, the offending lines and the totals are printed on standard output. Some common messages:

| Message | Usual cause |
|---|---|
| `opcode expected` | Misspelled or unknown instruction, or a label that isn't in column 1 |
| `undefined label` | A symbol is used but never defined or imported |
| `redefined label` | The same label is defined twice |
| `illegal address mode` | The instruction has no such addressing mode, such as `staa #1` or `clr <$80` |
| `address out of bounds` | Branch target too far away, or indexed offset outside 0–255 |
| `data out of bounds` | Value too large for a byte |
| `junk after operands` | Extra text after the operands; comments must start with `;` |
| `register used as identifier` | `A`, `B`, `D` or `X` used as a label |
| `absolute expression required` | A relocatable value where a constant is needed |

| Message | Usual cause |
|---|---|
| `no imports with binary file output` | `IMPORT` used together with `-b` |
| `short branch would do` | (warning) a `JSR`/`JMP` target is close enough for a `BSR`/`BRA` |

# Chapter 5

# Modules and Linking

## 5.1 Exporting and importing symbols

Each source file is assembled into an object module. By default, its symbols are private. To share them:

- In the module that defines a symbol, name it in an `EXPORT` (or `ENTRY`) directive.
- In each module that uses it, name it in an `IMPORT` directive.

```
; screen.s
        LOC     0

clrscn  ldx     #$4000
        ...
        rts
        EXPORT  clrscn
```

```
; main.s
        LOC     0
        IMPORT  clrscn
        ENTRY   start
start   jsr     clrscn
        rts
```

The linker reports `undefined symbol` for any import that no module exports.

Exactly one module should contain an `ENTRY` directive. The linker uses the first entry symbol it finds as the program's starting address, and fails with `no start symbol` if there is none.

## 5.2 Location counters and segments

The assembler keeps 16 independent location counters, numbered 0 to 15, selected with `LOC` or with `.TEXT`, `.DATA` and `.BSS`. Code assembled under different counters is kept apart, even if the source switches between them, and each module's counters start at 0.

The linker gathers all modules' counters into two segments:

| Counters | Segment |
|---|---|
| 0 and 1 | Text, placed at the `-T` address |

| Counters | Segment |
|---|---|
| 2 to 15 | Data, placed after the text, or at the `-D` address |

`.TEXT` selects counter 0, and `.DATA`, `.ROM` and `.BSS` all select counter 3. If counter 2 would cross a 256-byte page boundary, the linker moves it to the start of the next page.

Most MC6801 programs need only counter 0. Start each module with `LOC 0` (or `.TEXT`) to make this explicit.

## 5.3 Invoking ld01

```
ld01 [-dimMrsty] [-o outfile] [-T textaddr] [-D dataaddr] [-H heapsize]
     [-Ccrt] [-Llibdir] [-llib] [-Olibfile] file ...
```

| Option | Meaning |
|---|---|
| `-o outfile` | Output file name (default `a.out`) |
| `-T addr` | Text segment address |
| `-D addr` | Data segment address |
| `-H size` | Add *size* to the end of the program for `__heap_top` |
| `-Cname` | Link the start-up module `crtname.o` |
| `-lname` | Search the library `libname.a` |
| `-Ldir` | Add a library directory |
| `-Ofile` | Link *file*, looking in the library directories |
| `-d` | Write a raw memory image with no header or symbols |
| `-s` | Leave out the symbol table |
| `-m` | List the modules linked |
| `-M` | List all symbols with their values |
| `-r` | Combine the input objects into an archive instead of linking |
| `-i` | Mark the executable as having separate instruction and data spaces |
| `-t` | Trace modules as they are read (must come before the input files) |

The order of objects and libraries on the command line does not matter: the linker reads them all before resolving symbols.

Addresses and sizes are read like C numbers: hexadecimal with a `0x` or `$` prefix, octal with a leading `0`, and decimal otherwise. So `-T0x1000`, `-T'$1000'` and `-T4096` all mean `$1000` (quote `$` from the shell). Anything else, such as `-TE000`, or a value above `$FFFF`, is an error. `objcopy01` reads its `-T` and `-D` options the same way.

## 5.4 Output

The linker normally writes an a.out file: a 32-byte header (Appendix B), the text segment, the data segment and a symbol table. `objcopy01` converts this into the format your target needs (Chapter 8).

With `-d`, the output is a raw memory image containing just the text segment followed by the data segment, ready to be burned into an EPROM:

```
$ ld01 -d -T0xe000 -o rom.bin start.o main.o
```

## 5.5 Start-up modules

A *start-up module* is code that must run first: setting up the stack, clearing memory, then calling the main program. By convention it is called `crtname.o` (from "C run-time") and linked with `-Cname`:

```
; crtstart.s
        LOC     0
        IMPORT  main
        ENTRY   __start
__start lds     #$00ff
        jsr     main
        rts

$ as01 -o crtstart.o crtstart.s
$ ld01 -Cstart -T0x434c -o prog.aout main.o
```

## 5.6 Libraries

`-lname` links the needed members of the archive `libname.a`. An archive member is linked only if it defines a symbol that something else being linked uses. See Chapter 6 for how to build libraries.

Files named with `-C`, `-l` and `-O` are searched for in the `-L` directories (up to five), most recent first. There is no default library directory. If they are not found, the name is tried as given, in the current directory.

## 5.7 Linker symbols

The linker defines these symbols, which a program can `IMPORT`:

| Symbol | Value |
|---|---|
| `__btext`, `__etext` | Start and end of the text segment |
| `__bdata`, `__edata` | Start and end of the data segment |

| Symbol | Value |
|---|---|
| `__end` | End of the program, including uninitialized data |
| `__heap_top` | `__end` plus the `-H` value |

For example, a start-up module can clear uninitialized data between `__edata` and `__end`.

`__btext`, `__etext`, `__bdata`, `__edata` and `__end` are always written to the executable's symbol table (unless `-s` is used), where `objcopy01` finds the load addresses.

## 5.8 Link maps

`-m` prints a line for each module: `+` if it was linked, `-` if it is an archive member that was not needed. `-M` prints every symbol with the module that defines it, its value and its type:

```
$ ld01 -lnum -Cstart -T0x434c -m -M -o main.aout main.o screen.o
one             +libnum.a(one.o)
crtstart        +crtstart.o
main            +main.o
screen          +screen.o
                 one                    one  0  00004351  R
            crtstart               __start  0  00004380  R N
              screen                SCREEN  0  00004000  a
...
Total memory used: 000004ff
```

# Chapter 6

# Libraries

A library is an archive of object modules, created and maintained with `ar01`. The usual name is `libname.a`, so that it can be linked with `-lname`.

```
$ ar01 rc libnum.a one.o two.o three.o        # create or update
$ ar01 t libnum.a                             # list members
one.o
two.o
three.o
$ ar01 d libnum.a two.o                       # delete a member
$ ar01 x libnum.a one.o                       # extract a member
```

| Key | Operation |
|---|---|
| `r` | Replace or add members (creates the archive if necessary) |
| `q` | Append members without checking for duplicates |
| `d` | Delete members |
| `m` | Move members (to the end, or with `a`/`b` next to another member) |
| `t` | List members (`tv` for details) |
| `x` | Extract members |
| `p` | Print members to standard output |

Modifiers: `c` (create silently), `v` (verbose), `u` (replace only newer files), `o` (keep dates on extraction), `a`/`b`/`i` *posname* (position), and `s`, which adds a `__.SYMDEF` index of the symbols each member exports. The linker doesn't need the index, since it reads every member's symbols itself, but it does no harm.

Put each routine that may be needed separately in its own module, so that the linker includes only what a program uses.

# Chapter 7

# Inspecting Object Files and Executables

`objdump01` shows the header, segment sizes, symbol table and a hex dump of an a.out executable:

```
$ objdump01 main.aout
OBJECTFILE 'main.aout'
HLEN 32
CPU  m6801 BIG_ENDIAN
FLAGS: A_EXEC
a_entry  = 0x00004380
a_total  = 0x0000006c
a_syms   = 0x00000130

text    data    bss     dec     hex     filename
108     0       0       108     6c      main.aout

SYMBOLS
0000434c t msg_one
00004351 T one
...
TEXTSEG
: 4f 4e 45 0d 00 ce 43 4c  3c bd 43 a5 31 31 39 00  ONE...CL<.C.119.
...
```

`nm01` (or `objdump01 -n`) prints only the symbols, and `size01` (or `objdump01 -s`) only the sizes:

```
$ size01 main.aout
text    data    bss     dec     hex     filename
108     0       0       108     6c      main.aout
$ nm01 main.aout
0000434c t msg_one
00004351 T one
```

```
00004380 T __start
00004000 a SCREEN
...
```

In the symbol list, `T/t` are text symbols, `D/d` data symbols, `C/b` bss or common symbols and `a` absolute symbols. Upper case means global, lower case local, and `U` marks a symbol that is used but not defined. Symbol names in executables are cut to 8 characters.

The same tools work on object files made by `as01`, and on libraries made by `ar01`, which shows each module in turn. For an object file, `objdump01` shows the module's symbols, its size, and its code with the relocation records that `ld01` uses to fill in addresses:

```
$ objdump01 main.o
OBJECTFILE 'main.o'
MODULE  'main'
...
SYMS 6
SYM 0    00000000 ..E.T main
SYM 1    00000000 ...IT three
...
BYTECODE
SEG 0
: bd                                                    .
RELSZ 2
DW clrscn
...
$ nm01 main.o
00000000 T main
         U three
         U one
...
```

Here `DW clrscn` means that the two bytes after the `jsr` opcode (`bd`) are filled in with the address of `clrscn`. In object files, location counters 0 and 1 count as text and the others as data, as they are linked by `ld01`.

# Chapter 8

# Converting Executables

`objcopy01` turns an a.out file into a format that a loader, EPROM programmer, emulator or cassette interface can use:

```
objcopy01 [-O format] [-T textaddr] [-D dataaddr] [-n name] [-p]
          infile outfile
```

| Format | Output |
|---|---|
| `binary` | Raw image: text, then data (`-p` pads to put data at its address) |
| `srec` | Motorola S-records (S0 name, S1 data, S5 count, S9 entry point) |
| `ihex` | Intel hex, with the entry point in the end record |
| `wav` | Cassette audio: 8-bit mono WAV at 43200 Hz, Color BASIC/MC-10 format |
| `cas`, `c10` | The same cassette data as a byte stream, for emulators (`c10` matches the name used by MC-10 emulators) |

The load addresses come from the `__btext` and `__bdata` symbols that the linker writes to the executable, so you don't normally need to give them. Use `-T` and `-D` to override them, or when the executable was linked with `ld01 -s` and has no symbol table. Addresses are written the same way as for `ld01`: `0x434c` or `'$434c'` for hexadecimal, plain digits for decimal.

The entry point is taken from the executable header. `-n` sets the name stored in S-record headers and cassette files (up to 8 characters for cassettes); without it, the output file name in upper case is used.

```
$ objcopy01 -O srec -n ROM rom.aout rom.s19
$ objcopy01 -O wav -n GAME game.aout game.wav
$ objcopy01 -O srec -T 0xe000 -n ROM stripped.aout rom.s19
```

The cassette formats require the data segment to follow the text segment directly (no `-D`).

# Chapter 9

# A Worked Example

The `demo` directory contains a small modular program for the MC-10. It shows modules, a library and a start-up module working together. With the tools installed and on your `PATH`, build it with:

```
$ cd demo
$ make
```

## 9.1 Modules

`screen.s` provides two subroutines, `clrscn` and `prstring`, and exports them:

```
LOC 0

SCREEN  equ     $4000

clrscn  ldx     #SCREEN
        ldaa    #$40+'
        ldab    #$40+'
clrsc.1 std     ,x
        inx
        inx
        cpx     #(SCREEN+512)
        blt     clrsc.1
        rts

        EXPORT  clrscn

prstring
        tsx
        ldx     2,x
prstlop ldaa    ,x
        beq     prstxit
        inx
```

```
        pshx
        ldx     $ffde
        jsr     ,x
        pulx
        bra     prstlop
prstxit rts

        EXPORT  prstring
```

`prstring` takes a pointer to the string on the stack. `tsx` points X at the top of the stack, and `ldx 2,x` loads the argument, which is just above the return address.

## 9.2 A library

`one.s`, `two.s` and `three.s` each print a word. Here is `one.s`:

```
LOC 0

        IMPORT  prstring

msg_one fcb     'O,'N,'E,$0d,$00

one
        ldx     #msg_one
        pshx
        jsr     prstring
        ins
        ins
        rts

        EXPORT  one
```

They are assembled and collected into `libnum.a`:

```
$ as01 -o one.o one.s
$ as01 -o two.o two.s
$ as01 -o three.o three.s
$ ar01 r libnum.a one.o two.o three.o
```

## 9.3 The start-up module and main program

`crtstart.s` contains the entry point and calls `main`:

```
LOC 0

        IMPORT  main

        ENTRY   __start

__start
        jsr     main
        rts
```

`main.s` uses the routines from `screen.o` and the library:

```
LOC 0

        IMPORT  clrscn
        IMPORT  prstring

        IMPORT  one
        IMPORT  two
        IMPORT  three

main
        jsr     clrscn

        jsr     one
        jsr     two
        jsr     three
        rts

        EXPORT  main
```

## 9.4 Linking and converting

```
$ ld01 -lnum -Cstart -T0x434c -o main.aout main.o screen.o
$ objcopy01 -T0x434c -O wav main.aout main.wav
```

The linker pulls in `crtstart.o` because of `-Cstart`, and `one.o`, `two.o` and `three.o` from `libnum.a` because `main.o` uses them. Load `main.wav` on the MC-10 with `CLOADM` and run it with `EXEC`.

# Chapter 10

# Known Limitations

- **Symbol names are cut to 8 characters** in executables.
- **Warnings are off by default**; use `as01 -w-` or `.WARN 1`.

# Appendix A

# Instruction Set

The table lists the opcode (in hexadecimal) for each instruction in each addressing mode. It was produced by assembling every instruction with `as01`.

INH = inherent, IMM = immediate, DIR = direct, IDX = indexed, EXT = extended, REL = relative.

Instruction lengths: inherent 1 byte; direct, indexed and relative 2 bytes; extended 3 bytes; immediate 2 bytes, or 3 for `ADDD`, `CPX`, `CMPX`, `LDD`, `LDS`, `LDX` and `SUBD`.

| Mnemonic | INH | IMM | DIR | IDX | EXT | REL | Operation |
|---|---|---|---|---|---|---|---|
| **ABA** | 1B | | | | | | Add B to A |
| **ABX** | 3A | | | | | | Add B to X (unsigned) |
| **ADCA** | | 89 | 99 | A9 | B9 | | Add with carry to A |
| **ADCB** | | C9 | D9 | E9 | F9 | | Add with carry to B |
| **ADDA** | | 8B | 9B | AB | BB | | Add to A |
| **ADDB** | | CB | DB | EB | FB | | Add to B |
| **ADDD** | | C3 | D3 | E3 | F3 | | Add to D |
| **ANDA** | | 84 | 94 | A4 | B4 | | AND into A |
| **ANDB** | | C4 | D4 | E4 | F4 | | AND into B |
| **ASL** | | | | 68 | 78 | | Arithmetic shift left memory |
| **ASLA** | 48 | | | | | | Arithmetic shift left A |
| **ASLB** | 58 | | | | | | Arithmetic shift left B |
| **ASLD** | 05 | | | | | | Arithmetic shift left D |
| **ASR** | | | | 67 | 77 | | Arithmetic shift right memory |
| **ASRA** | 47 | | | | | | Arithmetic shift right A |
| **ASRB** | 57 | | | | | | Arithmetic shift right B |
| **BCC** | | | | | | 24 | Branch if carry clear |
| **BCS** | | | | | | 25 | Branch if carry set |
| **BEQ** | | | | | | 27 | Branch if equal |
| **BGE** | | | | | | 2C | Branch if greater or equal (signed) |
| **BGT** | | | | | | 2E | Branch if greater (signed) |
| **BHI** | | | | | | 22 | Branch if higher (unsigned) |
| **BHS** | | | | | | 24 | Branch if higher or same (= BCC) |
| **BITA** | | 85 | 95 | A5 | B5 | | Bit test A |
| **BITB** | | C5 | D5 | E5 | F5 | | Bit test B |
| **BLE** | | | | | | 2F | Branch if less or equal (signed) |
| **BLO** | | | | | | 25 | Branch if lower (= BCS) |

| Mnemonic | INH | IMM | DIR | IDX | EXT | REL | Operation |
|---|---|---|---|---|---|---|---|
| **BLS** | | | | | | 23 | Branch if lower or same (unsigned) |
| **BLT** | | | | | | 2D | Branch if less (signed) |
| **BMI** | | | | | | 2B | Branch if minus |
| **BNE** | | | | | | 26 | Branch if not equal |
| **BPL** | | | | | | 2A | Branch if plus |
| **BRA** | | | | | | 20 | Branch always |
| **BRN** | | | | | | 21 | Branch never |
| **BSR** | | | | | | 8D | Branch to subroutine |
| **BVC** | | | | | | 28 | Branch if overflow clear |
| **BVS** | | | | | | 29 | Branch if overflow set |
| **CBA** | 11 | | | | | | Compare B with A |
| **CLC** | 0C | | | | | | Clear carry |
| **CLI** | 0E | | | | | | Clear interrupt mask |
| **CLR** | | | | 6F | 7F | | Clear memory |
| **CLRA** | 4F | | | | | | Clear A |
| **CLRB** | 5F | | | | | | Clear B |
| **CLV** | 0A | | | | | | Clear overflow |
| **CMPA** | | 81 | 91 | A1 | B1 | | Compare A |
| **CMPB** | | C1 | D1 | E1 | F1 | | Compare B |
| **CMPX** | | 8C | 9C | AC | BC | | Compare X (= CPX) |
| **COM** | | | | 63 | 73 | | Complement memory |
| **COMA** | 43 | | | | | | Complement A |
| **COMB** | 53 | | | | | | Complement B |
| **CPX** | | 8C | 9C | AC | BC | | Compare X |
| **DAA** | 19 | | | | | | Decimal adjust A |
| **DEC** | | | | 6A | 7A | | Decrement memory |
| **DECA** | 4A | | | | | | Decrement A |
| **DECB** | 5A | | | | | | Decrement B |
| **DES** | 34 | | | | | | Decrement stack pointer |
| **DEX** | 09 | | | | | | Decrement X |
| **EORA** | | 88 | 98 | A8 | B8 | | Exclusive OR into A |
| **EORB** | | C8 | D8 | E8 | F8 | | Exclusive OR into B |
| **INC** | | | | 6C | 7C | | Increment memory |
| **INCA** | 4C | | | | | | Increment A |
| **INCB** | 5C | | | | | | Increment B |
| **INS** | 31 | | | | | | Increment stack pointer |
| **INX** | 08 | | | | | | Increment X |
| **JMP** | | | | 6E | 7E | | Jump |
| **JSR** | | | 9D | AD | BD | | Jump to subroutine |
| **LDAA** | | 86 | 96 | A6 | B6 | | Load A |
| **LDAB** | | C6 | D6 | E6 | F6 | | Load B |
| **LDD** | | CC | DC | EC | FC | | Load D |
| **LDS** | | 8E | 9E | AE | BE | | Load stack pointer |
| **LDX** | | CE | DE | EE | FE | | Load X |
| **LSL** | | | | 68 | 78 | | Logical shift left memory (= ASL) |
| **LSLA** | 48 | | | | | | Logical shift left A (= ASLA) |
| **LSLB** | 58 | | | | | | Logical shift left B (= ASLB) |
| **LSLD** | 05 | | | | | | Logical shift left D (= ASLD) |
| **LSR** | | | | 64 | 74 | | Logical shift right memory |
| **LSRA** | 44 | | | | | | Logical shift right A |
| **LSRB** | 54 | | | | | | Logical shift right B |
| **LSRD** | 04 | | | | | | Logical shift right D |

| Mnemonic | INH | IMM | DIR | IDX | EXT | REL | Operation |
|---|---|---|---|---|---|---|---|
| **MUL** | 3D | | | | | | Multiply A by B into D (unsigned) |
| **NEG** | | | | 60 | 70 | | Negate memory |
| **NEGA** | 40 | | | | | | Negate A |
| **NEGB** | 50 | | | | | | Negate B |
| **NOP** | 01 | | | | | | No operation |
| **ORAA** | | 8A | 9A | AA | BA | | OR into A |
| **ORAB** | | CA | DA | EA | FA | | OR into B |
| **PSHA** | 36 | | | | | | Push A |
| **PSHB** | 37 | | | | | | Push B |
| **PSHX** | 3C | | | | | | Push X |
| **PULA** | 32 | | | | | | Pull A |
| **PULB** | 33 | | | | | | Pull B |
| **PULX** | 38 | | | | | | Pull X |
| **ROL** | | | | 69 | 79 | | Rotate left memory |
| **ROLA** | 49 | | | | | | Rotate left A |
| **ROLB** | 59 | | | | | | Rotate left B |
| **ROR** | | | | 66 | 76 | | Rotate right memory |
| **RORA** | 46 | | | | | | Rotate right A |
| **RORB** | 56 | | | | | | Rotate right B |
| **RTI** | 3B | | | | | | Return from interrupt |
| **RTS** | 39 | | | | | | Return from subroutine |
| **SBA** | 10 | | | | | | Subtract B from A |
| **SBCA** | | 82 | 92 | A2 | B2 | | Subtract with carry from A |
| **SBCB** | | C2 | D2 | E2 | F2 | | Subtract with carry from B |
| **SEC** | 0D | | | | | | Set carry |
| **SEI** | 0F | | | | | | Set interrupt mask |
| **SEV** | 0B | | | | | | Set overflow |
| **STAA** | | | 97 | A7 | B7 | | Store A |
| **STAB** | | | D7 | E7 | F7 | | Store B |
| **STD** | | | DD | ED | FD | | Store D |
| **STS** | | | 9F | AF | BF | | Store stack pointer |
| **STX** | | | DF | EF | FF | | Store X |
| **SUBA** | | 80 | 90 | A0 | B0 | | Subtract from A |
| **SUBB** | | C0 | D0 | E0 | F0 | | Subtract from B |
| **SUBD** | | 83 | 93 | A3 | B3 | | Subtract from D |
| **SWI** | 3F | | | | | | Software interrupt |
| **TAB** | 16 | | | | | | Transfer A to B |
| **TAP** | 06 | | | | | | Transfer A to condition codes |
| **TBA** | 17 | | | | | | Transfer B to A |
| **TPA** | 07 | | | | | | Transfer condition codes to A |
| **TST** | | | | 6D | 7D | | Test memory |
| **TSTA** | 4D | | | | | | Test A |
| **TSTB** | 5D | | | | | | Test B |
| **TSX** | 30 | | | | | | Transfer stack pointer + 1 to X |
| **TXS** | 35 | | | | | | Transfer X - 1 to stack pointer |
| **WAI** | 3E | | | | | | Wait for interrupt |

# Appendix B

# File Formats

## B.1 Object files

`as01` writes object files in the Introl linkable binary format, described in the *Introl Linker and Loader Reference Manual*. They start with the two bytes `S1` (`$53 $31`).

## B.2 Executables

`ld01` writes a.out executables. All multi-byte fields are big-endian. The file starts with a 32-byte header:

| Offset | Size | Field | Contents |
|---|---|---|---|
| 0 | 2 | magic | `$01 $03` |
| 2 | 1 | flags | `$10` executable; `$20` separate instruction and data (`-i`) |
| 3 | 1 | cpu | `$1F` (MC6801) |
| 4 | 1 | header length | `$20` (32) |
| 5 | 1 | unused | 0 |
| 6 | 2 | version | 0 |
| 8 | 4 | text | Size of the text segment |
| 12 | 4 | data | Size of the data segment |
| 16 | 4 | bss | Size of uninitialized data |
| 20 | 4 | entry | Entry point address |
| 24 | 4 | total | Total memory used |
| 28 | 4 | syms | Size of the symbol table |

The header is followed by the text segment, the data segment and the symbol table. Each symbol table entry is 16 bytes: an 8-byte name (padded with zeros, not necessarily terminated), a 4-byte value, a 1-byte storage class, a 1-byte auxiliary count and a 2-byte type.

`ld01 -d` writes only the text and data segments, with no header or symbols.

# Appendix C

# Command Reference

This appendix contains the manual pages for the tools, as installed in `doc/man`.

## C.1 as01(1)

### NAME

as01 - assembler for the Motorola MC6801/MC6803

### SYNOPSIS

**as01** [**-gkuvw**] [**-b** *binfile*] [**-l** [*listfile*]] [**-m** [*listfile*]] [**-n** *name*] [**-o** *objfile*] [**-s** *symfile*] [**-t** *n*] [*srcfile*]

### DESCRIPTION

**as01** assembles one MC6801 source file. It writes a relocatable object file in Introl linkable binary format for **ld01**(1), an absolute binary image, or both. It can also write a listing and a symbol table.

The assembler makes two passes over the source. If *srcfile* is omitted, it reads standard input. Options and the source file may appear in any order. Each option must be a separate argument, and each output file may be given only once.

If there are errors, the object and binary files are deleted (unless **-k** is given) and the exit status is 1.

### OPTIONS

**-b *binfile*** Write an absolute binary image to *binfile*. Use **ORG** to set the load address. The image starts at the lowest address assembled and ends at the highest; any gaps between **ORG** areas are filled with zeros. **IMPORT**, **.GLOBL** and other external references are not allowed in this mode.

**-g** Put only global (exported and imported) symbols in the object file.

**-k** Keep the object and binary files even if there are errors.

**-l** [***listfile***] Write a listing to *listfile*, or to standard output if no file name follows.

**-m** [***listfile***] List every line of each macro expansion, including directives and lines skipped by conditionals. On its own, **-m** lists only the macro expansions; use **-l -m** *listfile* to get the full listing as well.

**-n** ***name*** Use *name* to derive the module name stored in the object file. By default the source file name is used. Any directory part and the last suffix are removed.

**-o** ***objfile*** Write a relocatable object file to *objfile*.

**-s** ***symfile*** Write a symbol table to *symfile*.

**-t** ***n*** Assemble **.TEXT** sections into location counter $n$+3 instead of location counter 0.

**-u** Treat undefined symbols as imported, as if each had been named in an **IMPORT** directive.

**-w** Disable warnings. This is the default; use **-w-** to enable them.

**-v** Print a version message and exit.

The flags **-g**, **-u** and **-w** are turned off by appending **-**, for example **-w-**.

## SOURCE FORMAT

Each line has the form

[*label*[**:**]] [*operation* [*operands*]] [**;** *comment*]

A label without a colon must start in column 1. A label followed by a colon may be indented. An operation must be preceded by white space unless it follows a label. A line whose first character is *****, **;** or **!** is a comment. A **;** or **!** starts a comment anywhere else on the line.

Symbols may contain letters, digits, __ and **.**, and must not start with a digit. They are case sensitive. Mnemonics, directives and register names are recognized in all upper case or all lower case.

### Constants

| | |
|---|---|
| **123** | decimal |
| **$7F**, **0x7F** | hexadecimal |
| **0177** | octal (leading zero) |
| **%1010** | binary |
| **'A** or **'A'** | character |

### Expressions

From lowest to highest precedence:

| | |
|---|---|
| = < > | comparison (true is -1, false is 0) |
| + - \| | add, subtract, bitwise or |
| * / & « » | multiply, divide, and, shifts |
| - + ~ | unary minus, plus, complement |

Parentheses group subexpressions. A ***** in operand position is the current location counter.

### Addressing modes

| | | |
|---|---|---|
| Inherent | **inx** | |
| Immediate | **ldaa #$10** | # prefix |
| Direct | **ldaa $80** | address in $00-$FF, chosen automatically |
| Extended | **ldaa $1234** | |
| Indexed | **ldaa 5,x** | unsigned 8-bit offset; **,x** means **0,x** |
| Relative | **bne loop** | branches only |

A < before an operand forces direct addressing and a > forces extended. Direct addressing is chosen only when the address is known in pass 1, so forward references use extended addressing. The read-modify-write instructions (**NEG**, **COM**, **LSR**, **ROR**, **ASR**, **ASL**/**LSL**, **ROL**, **DEC**, **INC**, **TST**, **CLR**) and **JMP** have no direct mode on the MC6801 and always use extended for non-indexed operands.

## DIRECTIVES

Alternative spellings are separated by /.

### Data

**FCB/.BYTE/DB/.DATA1** ***expr***[**,*expr...***] Define bytes.

**FDB/.WORD/DW/.SHORT/.DATA2** ***expr***[**,*expr...***] Define 16-bit words (big-endian).

**FCC/.ASCII** ***delim text delim*** Define a string. The first character is the delimiter. C escape sequences such as **\n** and **\0** are recognized.

**.ASCIZ** ***delim text delim*** Like **FCC**, followed by a zero byte.

**RMB/.BLKB/.SPACE** ***count***[**,*fill***] Reserve *count* bytes, filled with *fill* if it is given.

**.BLKW/.ZEROW** ***count*** Reserve *count* words.

### Symbols

***label* EQU *expr*or *label* = *expr*** Define a constant.

***label* SET *expr*** Define a symbol that may be redefined.

**EXPORT/PUBLIC/.DEFINE *sym*[,*sym...*]** Make symbols defined in this module visible to other modules.

**ENTRY *sym*** Like **EXPORT**, and also mark *sym* as an entry point. The linker uses the first entry point as the program's start address.

**IMPORT/EXTERN/.EXTERN/EXTRN *sym*[,*sym...*]** Declare symbols defined in another module.

**.GLOBL/.GLOBAL *sym*[,*sym...*]** Export the symbols if they are defined in this module, otherwise import them.

***label* COMM *size*or *label* LCOMM *size*** Define a common block of *size* bytes. **LCOMM** makes it local to the module.

**.COMM/.LCOMM *name*,*size*** The same, in Unix assembler syntax.

### Location

**LOC *n*** Switch to location counter *n* (0 to 15). The linker places counters 0 and 1 in the text segment and the others in the data segment.

**.TEXT, .DATA, .ROM, .BSS** Switch to location counter 0 (**.TEXT**) or 3 (the others).

**.SECT *name*** Switch to the named section (**.text**, **.data** or **.bss**).

**ORG *expr*** Set the location counter.

**BLOCK *expr and* ENDB** Assemble the lines in between at *expr*, then restore the previous location counter.

**.ALIGN *n*** Advance the location counter to a multiple of *n*.

**.EVEN** Advance the location counter to an even address.

### Conditional assembly

**IF *expr* / ELSEIF *expr* / ELSE / ENDIF** Assemble lines selectively; a non-zero expression is true.

**IFC *(str1,str2)* / ELSEIFC *(str1,str2)*** True if the two strings are identical.

### Macros

**MACRO *name ...* MEND** Define a macro. In the body, **?1**, **?2**, ... are replaced by the comma-separated arguments of the invocation. Invoke it as *name arg1*,*arg2*...

### Other

**INCLUDE/GET *file*** Assemble the contents of *file*.

**END** End of source; the rest of the file is ignored.

**FAIL** Report an error.

**.LIST, .MACLIST** [***n***] Adjust the listing and macro-listing counters. They only have an effect when listing is enabled with **-l** or **-m**.

**.WARN** [***n***] Add *n* (default -1) to the warning counter. Warnings are shown while the counter is 0 or more. It starts at -1, or at 0 with **-w-**, so **.WARN 1** turns warnings on and **.WARN** turns them off again.

## FILES

***objfile*** Introl linkable binary object (magic bytes **S1**).

## EXAMPLES

Assemble a module and produce a listing on standard output:

```
as01 -o screen.o -l screen.s
```

Assemble an absolute image with no linking step:

```
as01 -b prog.bin prog.s
```

## DIAGNOSTICS

Errors and warnings are shown in the listing under the offending line, with a caret marking the position. A summary of the error and warning counts is printed at the end. If no listing file was requested, the offending lines and the summary are printed on standard output.

## SEE ALSO

**ld01**(1), **ar01**(1), **objdump01**(1), **objcopy01**(1)

The mc6801-tools User's Manual (*mc6801-tools-manual.pdf*).

## HISTORY

Derived from the MC6809 configuration of **as09** in Bruce Evans' C compiler, as maintained in the dev86 package.

## C.2 ld01(1)

### NAME

ld01 - linker for MC6801 object files

### SYNOPSIS

**ld01** [**-dimMPrstyZ**] [**-o** *outfile*] [**-S** *script*] [**-A** *symbol=value*] [**-T** *textaddr*] [**-D** *dataaddr*] [**-H** *heapsize*] [**-C***crt*] [**-L***libdir*] [**-l***lib*] [**-O***libfile*] *file ...*

### DESCRIPTION

**ld01** combines object files produced by **as01**(1) and modules from archives made with **ar01**(1) into an executable. It resolves references between modules and relocates code and data to their final addresses.

Each *file* is an object file or an archive. A module is linked only if the program uses it: starting from the modules that define the entry point, **__main** or **start**, the linker includes every module that defines a symbol a linked module refers to. Archives may appear anywhere on the command line.

A linker script controls where everything goes: which parts of which modules go into each output section, the section addresses, and extra symbols. Without **-S**, the linker uses a built-in default script, which gives the layout described under SEGMENTS. **-P** prints it, as a starting point for your own. See LINKER SCRIPTS.

The program starts at the first symbol declared with the assembler's **ENTRY** directive. Linking fails with "no start symbol" if there is none.

By default the output is an a.out executable with a 32-byte header, followed by the text segment, the data segment and a symbol table. Use **-d** for a plain memory image instead. **objcopy01**(1) converts a.out files to S-record, Intel hex, cassette and other formats.

### OPTIONS

Single-letter flags may be turned off again by appending **-**, for example **-s-**.

**-o *outfile*** Write the output to *outfile* instead of **a.out**.

**-T *textaddr*** Set the address of the text segment. The value is read like a C number: hexadecimal with a **0x** or **$** prefix, octal with a leading **0**, and decimal otherwise, so **-T0x1000**, **-T'$1000'** and **-T4096** all mean $1000. (Quote **$** from the shell.) Anything else, such as **1000x**, or a value above $FFFF, is an error. The default is 0.

**-D *dataaddr*** Set the address of the data segment, written as for **-T**. Unless **-d** is given, this implies **-i**. By default the data segment directly follows the text segment.

**-H *heapsize*** Set the symbol **___heap_top** to the end of the program plus *heapsize* (written as for **-T**).

**-C*crt*** Link **crt***crt*.**o** as a start-up module, for example **-Cstart** for **crtstart.o**.

**-l*lib*** Search the archive **lib***lib*.**a.**

**-L*libdir*** Add *libdir* to the directories searched for **-C**, **-l** and **-O** files.

**-O*libfile*** Link *libfile*, looking for it in the search directories.

**-d** Write a headerless memory image: the text segment followed by the data segment, with no header or symbols. Implies **-s**.

**-i** Mark the executable as having separate instruction and data spaces.

**-m** List the modules linked on standard output. Each line shows the module name and its source file. + marks a linked module and **-** an archive member that was not needed.

**-M** Print the layout, each output section with its address and size and the input sections in it, then all symbols linked, with module, value and type, on standard output.

**-S *script*** Use the linker script in *script* instead of the default script.

**-A *symbol=value*** Define *symbol*, as if the script began with *symbol* = *value*;. The script can use it, and it takes the place of the script's **PROVIDE** for the same symbol.

**-P** Print the default linker script on standard output and exit. The script includes the values of **-T**, **-D**, **-H** and **-d**. Linking with it (**-S**) gives the same output as linking without a script. No input files are needed.

**-Z** Make it an error for the script not to place some input (see Orphans under LINKER SCRIPTS).

**-r** Instead of linking, write the input objects to *outfile* as an **ar01**(1) archive.

**-s** Do not include a symbol table in the output.

**-t** Trace the modules as they are read. Must come before the input files.

**-y** Write an extended symbol table.

**-v** Print a version message and exit.

## LIBRARY SEARCH

Files named by **-C**, **-l** and **-O** are searched for in the **-L** directories, most recently added first. There is no default library directory. If the file is not found in any of them, the name is opened as given, relative to the current directory. Up to five **-L** directories may be given.

## SYMBOLS

The linker defines these symbols, which programs may import:

**___btext, ___etext** Start and end of the text segment.

**___bdata, ___edata** Start and end of the data segment.

**___end** End of all data, including uninitialized data and common blocks.

**___heap_top** End of the program plus the **-H** value.

**___btext**, **___etext**, **___bdata**, **___edata** and **___end** are always written to the executable's symbol table, unless **-s** is given. **objcopy01**(1) uses them to find the load addresses.

## LINKER SCRIPTS

A script is a list of commands. Comments are written /* **...** */ or start with #. For example:

```
ENTRY(reset)

SECTIONS
{
    .text 0xE000 :
    {
        crt0.o(LOC0)
        *(LOC0 LOC1)
        *(COMMON0 COMMON1)
    }
    .data 0x0080 :
    {
        *(LOC*)
        *(COMMON)
    }
    __stack_top = 0x00FF;
}
```

### Commands

**ENTRY(*symbol*)** Start the program at *symbol*, before any **ENTRY** in the object files.

**OUTPUT_FORMAT(*format*)** **aout** (the default); **binary**, a memory image as with **-d**, covering the load addresses from the lowest to the highest used; **srec**, Motorola S-records; or **ihex**, Intel hex. S-records and Intel hex have one run of records for each section written to the output, at its load address, without the gaps between sections, and the entry point in the end record. The S-record header holds the output file's name.

**MEMORY *{* name *[(*attributes*)]* : ORIGIN = *expr*, LENGTH = *expr ...* }** Define memory regions, such as ROM and RAM. The attributes, such as **rx**, are only shown in the **-M** layout.

**FILL(*byte*)** The byte for gaps, where a section doesn't set its own (default 0).

**ASSERT(*expression*, "*message*")** Stop with *message* if *expression* is 0 once the layout is complete. The modules that define its symbols are linked.

**INCLUDE "*file*"** Read another script here (also allowed inside **SECTIONS**).

**SECTIONS *{ ... }*** Output sections and symbol assignments, in order.

***symbol* = *expression*;** Define a symbol.

**PROVIDE(*symbol* = *expression*);** Define it only if a linked module imports it and no module defines it.

**HIDDEN(*symbol* = *expression*);** Define a symbol that only the script can use.

### Output sections

```
name [address] [(NOLOAD)] : [AT(lma)] [ALIGN(n)] [SUBALIGN(n)]
    { statements } [> region] [AT> region] [= fill]
```

Without an *address*, a section placed in a region (*> region*) starts at the region's next free address, and other sections start at the current address (**.**), which is where the previous section ended. It's an error if a section doesn't fit in its region.

A section is stored at its load address, which is the same as its address unless **AT** gives another one, or **AT>** allocates it from a region. For example, initialized data that runs in RAM can be stored in ROM after the code, and copied by the start-up code. Separate load addresses need the **binary**, **srec** or **ihex** output format.

**NOLOAD** sections take up addresses but aren't written to the output. **= fill** sets the byte for gaps in the section. **ALIGN** rounds the start up to a multiple of *n*. **SUBALIGN** pads each input section in it to a multiple of *n* bytes.

Statements inside a section, in order:

***file*(*sections*)** Add the matching input sections.

**ONEPAGE *{ statements }*** Keep the contents within one 256-byte page: if they would end on a later page than they start, they move to the start of the next page.

***symbol* = *expression*;, *PROVIDE*, *HIDDEN*** Define symbols, often using **.**, the current address.

**. = *expression*;** Move the current address forward. The gap is filled with the fill byte.

**KEEP(*file*(*sections*))** Add input sections like *file*(*sections*), and link their modules even if nothing refers to them.

**BYTE(*expression*); WORD(*expression*);** Put a byte, or a 16-bit word (high byte first), into the section.

**FILL(*byte*);** Use *byte* for the gaps that follow.

**ASSERT(*expression*, "*message*");** As at the top level, but with **.** as here.

### Input sections

Each linked module has two input sections for each location counter *n*: **LOC***n,* its code and data, and **COMMON***n,* its common blocks. **.text** means **LOC0**, **.data** means **LOC3**, and **COMMON** means all the common blocks.

*file* matches the object file name, with or without its directory; archive members are named *archive***:***member.* Both *file* and the section names may use the wildcards ***** and **?**. Each input section goes to the first statement that matches it. A statement adds its input sections in module order, and each module's in counter order.

### Orphans

Input that no statement matches is added, with a warning, at the end of the last output section for its kind (counters 0 and 1, or the others) that takes input from all files (*****), or else the last section for its kind. **-Z** makes it an error.

### Expressions

Expressions use C operators and numbers written as for **-T**. A **K** suffix multiplies by 1024. The functions are **ADDR(***section***), SIZEOF(***section***), LOADADDR(***section***), ALIGN(***n***)** (the current address rounded up), **ALIGN(***expression***,** *n***), DEFINED(***symbol***), ORIGIN(***region***), LENGTH(***region***), MAX** and **MIN**. Symbols may be used before they're defined; the linker repeats the layout until nothing changes.

### Symbols

Symbols defined by a script are absolute. They satisfy imports from any location counter. A plain assignment to a symbol that a module also defines is an error.

For each output section, the linker also provides **___***name***__start, ___***name***__end** and **___***name***__load** (its load address), with the section name's leading dot dropped, if a module imports them. For example, start-up code can copy **.data** from **___data__load** to **___data__start**.

### a.out output

The output sections up to the first one that can take input from location counters 2 to 15 form the a.out text part, and the rest the data part. The output is marked as having separate instruction and data spaces if the data part doesn't start where the text part ends. Common blocks at the end of the data part are its uninitialized data. The header's total memory size reaches **___heap__top** if the script sets it.

## SEGMENTS

This is the layout of the default script. The assembler has 16 location counters. Counters 0 and 1 are linked into the text segment, and counters 2 through 15 into the data segment. If counter 2

would cross a 256-byte page boundary, it is moved to the start of the next page. **.TEXT** selects counter 0, and **.DATA** and **.BSS** select counter 3.

### EXAMPLES

Link a start-up module, two objects and a library, loading at $434C:

**ld01 -Cstart -lnum -T0x434c -o main.aout main.o screen.o**

Produce a raw memory image for an EPROM at $E000:

**ld01 -d -T0xe000 -o rom.bin start.o prog.o**

Start a linker script from the default layout, edit it, and use it:

ld01 -P -T0x434c > mc10.ld ld01 -S mc10.ld -Cstart -lnum -o main.aout main.o screen.o

### DIAGNOSTICS

Undefined symbols are reported as "undefined symbol: *name*". The exit status is 0 on success and non-zero on error.

### SEE ALSO

**as01**(1), **ar01**(1), **objdump01**(1), **objcopy01**(1)

The mc6801-tools User's Manual (*mc6801-tools-manual.pdf*).

### BUGS

Symbol names in the a.out symbol table are cut to 8 characters.

### HISTORY

Derived from **ld86** in Bruce Evans' C compiler, as maintained in the dev86 package.

## C.3 ar01(1)

### NAME

ar01 - create and maintain libraries of MC6801 object files

## SYNOPSIS

**ar01** [**-**]*key* [*posname*] *archive* [*file...*]

## DESCRIPTION

**ar01** maintains archives (libraries) of object files produced by **as01**(1). **ld01**(1) searches archives with its **-l** option and links only the members that are needed. The archive format is the standard Unix **!<arch>** format.

*key* is one operation letter, optionally combined with modifier letters. The leading **-** is optional.

## OPERATIONS

**r** Replace the named files in the archive, or add them at the end if they are not already there. Creates the archive if it does not exist.

**q** Quick append: add the named files at the end without checking for existing members.

**d** Delete the named members.

**m** Move the named members, by default to the end of the archive.

**t** List the members, or only the named ones.

**x** Extract the named members into the current directory, or all members if none are named.

**p** Write the contents of the named members (or all members) to standard output.

## MODIFIERS

**a** With **r** or **m**, place the files after member *posname*.

**b, i** With **r** or **m**, place the files before member *posname*.

**c** Do not report when a new archive is created.

**o** On extraction, give files the modification time recorded in the archive.

**u** With **r**, replace only members older than the named files.

**v** Verbose: report each file added, replaced or extracted, and give a long listing with **t**.

**l** Accepted and ignored.

**s** Add or update a **___.SYMDEF** member, an index of the symbols that each member exports, including common blocks. Once an archive has an index, later changes to the archive keep it up to date. **ld01**(1) does not need the index; it reads every member's symbols directly. Members that are not object files are reported and left out of the index.

### EXAMPLES

Create a library from three modules:

**ar01 rc libnum.a one.o two.o three.o**

List its contents in detail:

**ar01 tv libnum.a**

Link against it:

**ld01 -Cstart -lnum -T0x434c -o main.aout main.o**

### SEE ALSO

**as01**(1), **ld01**(1), **objdump01**(1)

The mc6801-tools User's Manual (*mc6801-tools-manual.pdf*).

### HISTORY

Derived from the GNU **ar** used in the dev86 package.

## C.4 objdump01(1)

### NAME

objdump01, nm01, size01 - display information about MC6801 object files and executables

### SYNOPSIS

**objdump01** [**-n**|**-s**] [**-o**] *file ...*
**nm01** [**-o**] *file ...*
**size01** *file ...*

### DESCRIPTION

**objdump01** displays the contents of object files produced by **as01**(1), archives of them made by **ar01**(1), and a.out executables produced by **ld01**(1). **make install** also installs it as **nm01** and **size01**, which behave as if **-n** or **-s** had been given. The mode is chosen from the first letter of the program name.

By default **objdump01** prints for each executable:

- the header: header length, CPU (m6801), flags, entry point (**a_entry**), total memory size (**a_total**) and symbol table size;
- the segment sizes;
- the symbol table;
- a hex and ASCII dump of the text and data segments.

For each object file, or each object file in an archive, it prints:

- the module name and the size of each location counter (**SEG0**, **SEG3**, ...);
- the symbol table, with flags for exported (**E**), imported (**I**), entry point (**N**) and common (**C**) symbols;
- the text, data and bss sizes;
- the module's code, with the relocation records that **ld01**(1) uses to fill in addresses, such as **DW clrscn** for a 2-byte reference to the symbol **clrscn**.

In object files, location counters 0 and 1 count as text and the others as data, as they are linked by **ld01**(1).

## OPTIONS

**-n** Print only the symbol table, like **nm**(1). Each line gives the value, a type letter and the name.

**-s** Print only the text, data and bss sizes, in decimal, followed by the total in decimal and hexadecimal, like **size**(1). If several files are given, a total line is added.

**-o** In **-n** mode, prefix each line with the file name instead of printing a heading for each file.

## SYMBOL TYPES

| | |
|---|---|
| **T**, **t** | text segment symbol (global, local) |
| **D**, **d** | data segment symbol (global, local) |
| **C**, **b** | bss or common symbol (global, local) |
| **a** | absolute symbol |
| **U** | undefined |
| **?** | other |

### EXAMPLES

```
$ size01 main.aout
text    data    bss dec hex filename
108 0   0   108 6c  main.aout

$ nm01 main.aout
0000434c t msg_one
00004351 T one
00004380 T __start
00004000 a SCREEN
...

$ nm01 main.o
00000000 T main
         U three
         U one
...
```

### SEE ALSO

**ld01**(1), **objcopy01**(1), **as01**(1)

The mc6801-tools User's Manual (*mc6801-tools-manual.pdf*).

### BUGS

The text and data segments are shown as hex dumps, not disassembled.

Symbol names in executables are cut to 8 characters.

Unknown options are silently ignored.

### HISTORY

Derived from **objdump86** in the dev86 package, by Greg Haerr and others.

## C.5 objcopy01(1)

### NAME

objcopy01 - convert MC6801 executables to loadable formats

### SYNOPSIS

**objcopy01** [**-O** *format*] [**-T** *textaddr*] [**-D** *dataaddr*] [**-n** *name*] [**-p**] *infile outfile*

## DESCRIPTION

**objcopy01** reads an a.out executable produced by **ld01**(1) and writes its text and data segments to *outfile* in one of several formats for loaders, EPROM programmers and cassette interfaces. The input must have an a.out header, so do not link it with **ld01 -d**.

The program's entry point is taken from the a.out header. The load addresses are taken from the **___btext** and **___bdata** symbols that **ld01**(1) writes to the symbol table. **-T** and **-D** override them. An executable linked with **ld01 -s** has no symbols, so give **-T** for it.

**ld01**(1) can also write S-records, Intel hex and binary images directly, with **OUTPUT_FORMAT** in a linker script. That handles programs spread over several areas of memory and separate load addresses, which a.out files can't hold.

## OPTIONS

**-O** ***format*** Output format (default **binary**). See FORMATS.

**-T** ***textaddr*** Load address of the text segment. The default is the value of **___btext**. The value is read like a C number: hexadecimal with a **0x** or **$** prefix, octal with a leading **0**, and decimal otherwise, so **-T0x1000**, **-T'$1000'** and **-T4096** all mean $1000. (Quote **$** from the shell.) Anything else, such as **1000x**, or a value above $FFFF, is an error. This is the same as **ld01 -T**.

**-D** ***dataaddr*** Load address of the data segment, for formats that can place it separately. The default is the value of **___bdata**; if there are no symbols, the data segment follows the text segment.

**-n** ***name*** Name to store in the output. This is the S0 header record name for **srec**, the cassette file name for **cas**/**c10** and **wav** (up to 8 characters, padded with spaces). The default is the output file name in upper case, including any directory and suffix, so it is usually best to give **-n** explicitly.

**-p** With **binary**, pad the gap between the text and data segments so that the data appears at its correct offset.

## FORMATS

The format name may be abbreviated to its first three letters.

**binary** Raw memory image: the text segment followed by the data segment.

**srec** Motorola S-records: an S0 header containing the name, S1 data records of 16 bytes, an S5 record count and an S9 record with the entry point.

**ihex** Intel hex: 16-byte data records and an end record holding the entry point.

**wav** Cassette audio as an 8-bit mono WAV file at 43200 Hz, in the Color BASIC tape format. This format is also used by the Tandy MC-10, which loads it with **CLOADM**. Text and data must be contiguous.

**cas, c10** The same cassette data as **wav**, as a byte stream for emulators. **c10** is another name for the same format, matching the **.c10** files used by MC-10 emulators.

## EXAMPLES

Make an MC-10 cassette image:

**objcopy01 -O wav -n MAIN main.aout main.wav**

Make an S-record file from an executable linked with **ld01 -s -T0xe000**, which has no symbols to give the load address:

**objcopy01 -O srec -T 0xe000 -n ROM rom.aout rom.s19**

## DIAGNOSTICS

"*format* load address is zero (use -T)" means the input has no symbol table (or was linked at address 0); give **-T**. "*format* data load address is zero (use -D)" is the same for a separate data segment.

## SEE ALSO

**ld01**(1), **objdump01**(1), **as01**(1)

The mc6801-tools User's Manual (*mc6801-tools-manual.pdf*).

## AUTHOR

John W. Linville.